

Vorlesung

Theoretische Informatik

(01 Einleitung)

Alois Heinz
Hochschule Heilbronn,
Max-Planck-Str. 39, 74081 Heilbronn
heinz@hs-heilbronn.de

10/13. März 2026

Organisatorisches

- Gemeinsame Veranstaltung SEB4, AI6, MIB5
- Einführung: 10/13.03.2026 um 09:45 Uhr in A211/F138
- Vorlesung: F235/F138, Di/Fr 09.45–11.15 Uhr
- Übungen: F235/F138, Di/Fr 11.30–13.00 Uhr
- Prüfung: Klausur (90 Min.) für benoteten Leistungsnachweis

Literatur

Vossen, Witt: Grundlagen der Theoretischen Informatik mit Anwendungen, Springer Vieweg; ISBN: 978-3-8348-1770-9

Themen der Vorlesung

Theorie der Berechenbarkeit

Welche Funktionen sind wann berechenbar?
Gibt es nicht berechenbare Funktionen? ...

Komplexitätstheorie

Wie groß ist der Aufwand zum Lösen von Problemen?
Kann die Lösung an zu großen Aufwand scheitern? ...

Automatentheorie

Welche Automaten-Modelle können Berechnungen modellieren?
Gibt es minimale Modelle, die bereits ausreichend sind? ...

Theorie der formalen Sprachen

Welche Grammatiken können Rechenausdrücke oder Programme in einer Programmiersprache beschreiben? Gibt es Zusammenhänge zwischen den Grammatiken und Automaten-Modellen? ...

Ziel: Antworten auf die gestellten Fragen finden, Automaten und Formale Sprachen kennenlernen, Komplexitäten abschätzen lernen (auch in Vorlesung Algo)

Einige Antworten der Vorlesung

Theorie der Berechenbarkeit: Es gibt nicht berechenbare Funktionen; es gibt überhaupt mehr Funktionen als berechnende Algorithmen ...

Komplexitätstheorie: Es gibt Probleme, bei denen die Zeit zum Finden einer Lösung polynomiell von der Problemgröße abhängt; bei anderen Problemen wird (vermutlich) immer exponentiell viel Zeit gebraucht ...

Automatentheorie

Verschiedene unterschiedlich mächtige Automaten-Modelle modellieren Berechnungen; die einfacheren Modelle sind effizienter aber weniger mächtig; verschiedene bereits universelle Modelle sind noch sehr einfach aufgebaut ...

Theorie der formalen Sprachen: Es gibt eine Hierarchie unterschiedlich mächtiger Grammatiken, die auf gewisse Weise den Automaten-Modellen entsprechen

Erkenntnis: Die Theoretische Informatik liefert interessante Antworten sowie Rüstzeug und Methoden zur Analyse und Lösung von sehr praktischen Problemen.

Historischer Abriss der Berechenbarkeitstheorie

- ~ 800 [Ibn Musa Al-Chwarizmi](#) (persischer Mathematiker und Astronom) schreibt Lehrbuch über das Lösen von algebraischen Gleichungen
- ~ 1300 [Raimundus Lullus](#) (katalanischer Philosoph) sucht in *Ars Magna* Methode zur Lösung aller mathematischen Probleme
- 1888 [Richard Dedekind](#): primitiv-rekursive Funktionen
- 1928 [Wilhelm Ackermann](#): berechenbare, nicht primitiv-rekursive Funktion
- bis 1931 Allgemeine Ansicht: Jedes hinreichend genau formulierte mathematische Problem kann gelöst werden. [David Hilbert](#) sucht nach allgemeinem Lösungsverfahren.
- 1931 [Kurt Gödel](#): In einem bestimmten formalen System gibt es kein Verfahren, jedes formulierbare Theorem als wahr oder falsch nachzuweisen (Unvollständigkeitssatz). Erste Algorithmenpräzisierung.
- um 1936 Verschiedene Präzisierungen des Algorithmenbegriffs. λ -Kalkül ([Church](#)), μ -rekursive Funktionen ([Kleene](#)), (allgem.) rekursive Funktionen ([Gödel](#), [Herbrand](#)), TURING-Maschinen ([Alan Turing](#))

Die Unlösbarkeit des Halteproblems

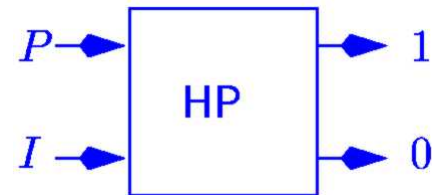
Als *Halteproblem (HP)* bezeichnet man die Aufgabe zu entscheiden, ob ein gegebener Algorithmus (ein Programm) nach endlicher Zeit hält (eine Lösung produziert) oder nicht.

Entscheidungsproblem HP:

Gegeben: Programm P , Input I

Frage: Hält $P(I)$?

Antwort:
$$\begin{cases} 1 & \text{falls } P(I) \text{ hält} \\ 0 & \text{falls } P(I) \text{ nicht hält} \end{cases}$$

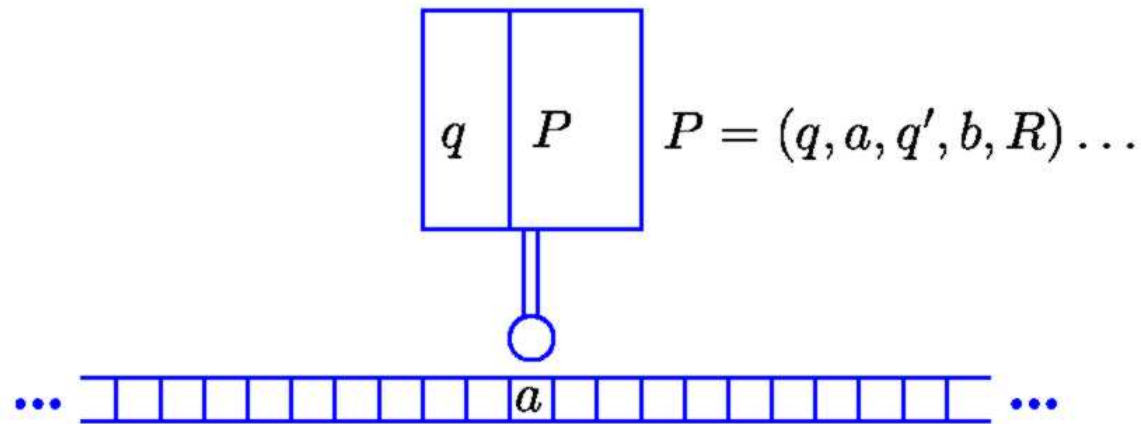


Satz. *Es gibt kein Verfahren, welches das Halteproblem für beliebige P und I entscheiden kann.*

Bem.: Aussage gilt für beliebige Rechnermodelle, sofern sie genügend komplex sind.

Rechnermodelle, Algorithmen, Turingmaschine

Turingmaschine: (A. Turing, 1936)



Input, Output: Band

Programm: Endlicher Text im Programmspeicher

These. [Church] *Die Klasse der Turing-berechenbaren Funktionen stimmt überein mit der Klasse der im intuitiven Sinne berechenbaren Funktionen.*

(Äquivalent zu Turing-berechenbar: While-berechenbar, Goto-berechenbar, μ -rekursiv)

Bem.: Es gibt eine Universelle Turingmaschine U : Ihr Input ist ein codiertes Turingprogramm P und weitere Daten I auf dem Band; U simuliert die Berechnungen von P auf I .

While-Programme

Variablen für Daten, Programme

Elementare Befehle

Zuweisungen, Vergleiche, Arithmetik, Input, Output

Kontrollstrukturen

- Komposition (Hintereinanderausführung, Programmaufruf)

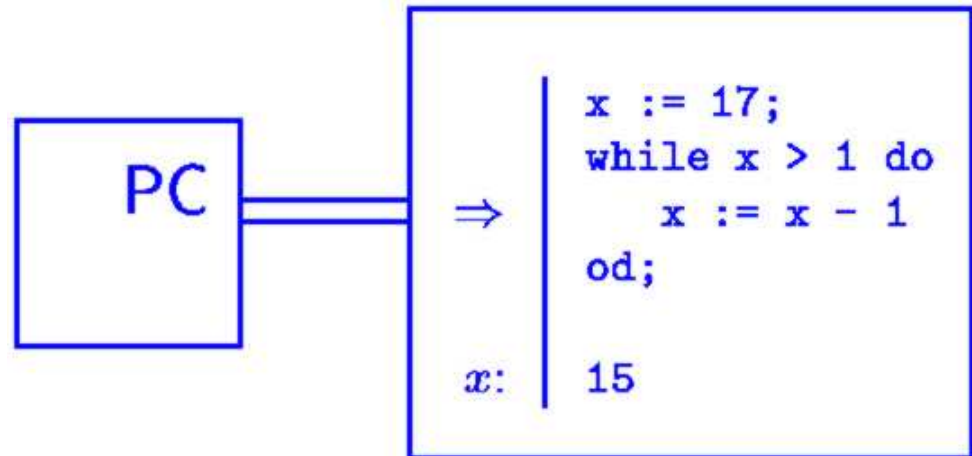
```
xyz := proc(x, y, z)
  return(x + y + z)
end;
xyz(a, b, c);
```

- Selektion (Fallunterscheidung)

```
if modp(x, 2) = 0
  then x := x/2
  else x := 3*x+1
fi;
```

- Iteration

```
while x > y do
  x := x-y
od;
```



Eigenschaften von Algorithmen

Unabhängig vom Rechnermodell gilt:

- Algorithmen werden durch Programme beschrieben
- Programme sind Texte fester Länge über endlichem Alphabet
- Alle gültigen Programme lassen sich auflisten (abzählen, mit Nummer identifizieren):

$$P_0, P_1, P_2, P_3, P_4, P_5, P_6, \dots$$

Satz. *Es gibt mehr Funktionen $f : \mathbb{N} \rightarrow \mathbb{N}$ als berechnende Programme.*

Beweis. Technik: **Diagonalisierung**

	0	1	2	3	4	5	...	n
f_{P_0}	③	3	3	3	3	3	...	
f_{P_1}	1	②	3	4	5	6	...	
f_{P_2}	0	$\perp$	①	$\perp$	$\perp$	$\perp$	...	
$\vdots$								$\vdots$
$f_d :$	4	3	0	.	.	.		

$f_d(i) = 0$, falls $f_{P_i}(i) = \perp$ und $f_d(i) = 1 + f_{P_i}(i)$ sonst $\Rightarrow f_d$ nicht berechenbar! $\square$

Wünschenswerte Eigenschaften von Algorithmen

- Korrektheit (partielle Korrektheit, Terminierung)
- Effizienz

Bsp.: `collatz:= proc(n)`
 `if n < 2 then 0`
 `elif n :: even`
 `then 1 + collatz(n/2)`
 `else 1 + collatz(3*n+1)`
 `fi`
`end:`

n	$\text{collatz}(n)$	Folge von n bis 1
7	16	7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
8	3	8, 4, 2, 1
27	111	27, 82, 41, 124, ..., 8, 4, 2, 1
$3^{50000} - 1$	567858	...

Stoppt die Berechnung von $\text{collatz}(n)$ immer?

Antwort: unbekannt

Programm zur Entscheidung der Terminierung

Gibt es ein Programm halt mit

$$\text{halt}(P, I) = \begin{cases} 1 & \text{falls } P(I) \text{ hält} \\ 0 & \text{sonst} \end{cases}$$

Antwort: Nein, das Halteproblem ist beweisbar unentscheidbar!

Unentscheidbarkeitsbeweis des Halteproblems

Beweis durch Widerspruch. Annahme: $\text{halt}(P, I)$ existiert.

Dann sei:

```
D := proc(P)
    if halt (P, P) = 1
    then while true do od
    else return (64)
    fi
end;
```

Es gilt:

$P(P)$ hält $\Rightarrow \text{halt}(P, P) = 1 \Rightarrow D(P)$ hält nicht

$P(P)$ hält nicht $\Rightarrow \text{halt}(P, P) = 0 \Rightarrow D(P) = 64$

Berechne $D(D)$: (Ersetze oben P durch D)

$D(D)$ hält $\Rightarrow \text{halt}(D, D) = 1 \Rightarrow D(D)$ hält nicht †

$D(D)$ hält nicht $\Rightarrow \text{halt}(D, D) = 0 \Rightarrow D(D) = 64$ †

Annahme führt zu Widerspruch (†), ist also falsch. $\square$

Bem: Techniken: Diagonalisierung und Selbstanwendung

Entscheidbare Teilprobleme von HP

Zeitbeschränktes Halteproblem

Hält $P(I)$ innerhalb einer gegebenen Anzahl elementarer Schritte?

Simuliere die Berechnung von $P(I)$ unter Zählung der Schritte. Stoppt die Berechnung vor Erreichen der Grenze, ist die Antwort positiv, sonst negativ.

Platzbeschränktes Halteproblem

Hält $P(I)$ während eine gegebene Speichergröße nicht überschritten wird?

Beträgt der zulässige Gesamtspeicher zur Ausführung von P maximal n Bit, dann gibt es höchstens 2^n verschiedene Speicherzustände. Die auftretenden Zustände können protokolliert, Speicherüberschreitung und Zustandswiederholung festgestellt werden. In letzteren Fällen ist die Antwort negativ, bei früherem Stopp positiv.

Halteproblem für spezielle Algorithmenklassen

Hält $P(I)$ wobei P aus einer beschränkten Klasse von Problemen ist?

Auswirkungen der Unentscheidbarkeit des HP

Die Unentscheidbarkeit des HP weist wegen der Churchschen These auf prinzipielle Schranken des menschlichen Geistes hin.

Wäre das HP entscheidbar, könnte man

- die Terminierung beliebiger Programme entscheiden, somit
- manche bislang unbewiesene mathematische Vermutung beweisen

Bsp.: Goldbachsche Vermutung: Jede gerade Zahl ≥ 4 lässt sich als Summe zweier Primzahlen darstellen.

Man can never eliminate the necessity of using his own cleverness, no matter how cleverly he tries. (Paul C. Rosenbloom; $\rightarrow$)

Weitere Auswirkungen der Unentscheidbarkeit des HP

Übertragung der Unentscheidbarkeit von HP auf andere Probleme durch *Reduktion*

Zehntes Hilbertsches Problem: Finde für multivariate Polynome mit ganzzahligen Koeffizienten ganzzahlige Nullstellen.

$$f(x, y, z) = 6x^3yz^2 + 3xy^2 - x^3 - 10 \text{ hat Lösung } (5, 3, 0)$$

Postsches Korrespondenzproblem: Finde für Wortpaare (a_i, b_i) Indexfolgen $i_1, \dots, i_m$ mit $a_{i_1} \dots a_{i_m} = b_{i_1} \dots b_{i_m}$

$$\frac{bab}{a}, \frac{a}{aba}, \frac{ab}{b}, \frac{ba}{b}, \frac{aba}{ab} \text{ hat Lösung } 2, 4, 1, 3, 2$$

Programmäquivalenzproblem: Bestimme ob zwei gegebene Programme dieselbe Funktion berechnen.

Datenkomprimierungsproblem: Bestimme das kürzeste Programm, das einen gegebenen Text ausgibt.

Viruserkennungsproblem

Probleme aus vielen Bereichen: Automatentheorie, Chaostheorie, Kombinatorik, Operation Research, Statistik, Physik, Compilerbau, Knotentheorie, Logik, Mengentheorie, Topologie, ...

Zusammenfassung

Durch Diagonalisierung und Selbstanwendung wurde gezeigt, dass für beliebige Programme nicht entschieden werden kann, ob sie jemals anhalten oder nicht.

Wegen der Unentscheidbarkeit des Halteproblems müssen viele interessante Problemstellungen auf Dauer und viele andere zumindest auf begrenzte Zeit ungelöst bleiben.

(Selbst viele prinzipiell lösbaren Probleme bleiben ungelöst wegen des nicht zu erbringenden Aufwands.)

⇒ Die Zukunft der Informatiker, Mathematiker, ...
bleibt auf jeden Fall interessant und beschäftigungsreich.